

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

**JEEVAN RAO JR
192524495**

1. Relational Algebra Operations

Selection (σ):

```
[  
\sigma_{Salary>50000}(Employee)  
]
```

Projection (π):

```
[  
\pi_{EmpName}(Employee)  
]
```

Union ($\cup$):

```
[  
(\sigma_{DeptID='IT'}(Employee)) \cup (\sigma_{DeptID='HR'}(Employee))  
]
```

Difference ($-$):

```
[  
Employee - (\sigma_{DeptID='IT'}(Employee))  
]
```

Cartesian Product ($\times$):

```
[  
Employee \times Department  
]
```

Join ($\bowtie$):

```
[  
Employee \bowtie_{Employee.DeptID=Department.DeptID} Department  
]
```

2. SQL DDL Statements

```
CREATE TABLE DEPARTMENT (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE STUDENT (  
    SID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    DOB DATE NOT NULL,  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)  
);
```

3. GROUP BY, HAVING and Aggregate Functions

```
SELECT COUNT(*) FROM Sales;
```

```
SELECT SUM(Amount) FROM Sales;
```

```
SELECT AVG(Amount) FROM Sales;
```

```
SELECT MAX(Amount) FROM Sales;
```

```
SELECT MIN(Amount) FROM Sales;
```

```
SELECT Region, SUM(Amount)  
FROM Sales  
GROUP BY Region;
```

```
SELECT Region, SUM(Amount)  
FROM Sales  
GROUP BY Region  
HAVING SUM(Amount) > 100000;
```

```
MySQL 8.0 Command Line Cli x + - x
+-----+
| 17000.000000 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT MAX(Amount) FROM Sales;
+-----+
| MAX(Amount) |
+-----+
| 50000.00 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT MIN(Amount) FROM Sales;
+-----+
| MIN(Amount) |
+-----+
| 1000.00 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT Region, SUM(Amount)
-> FROM Sales
-> GROUP BY Region;
+-----+
| Region | SUM(Amount) |
+-----+
| South | 52000.00 |
| North | 1000.00 |
| East | 15000.00 |
+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT Region, SUM(Amount)
-> FROM Sales
-> GROUP BY Region
-> HAVING SUM(Amount) > 100000;
```

4. Correlated Sub-query

```
SELECT EmpName, Salary
FROM Employee E
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employee
    WHERE DeptID = E.DeptID
);
```

```
MySQL 8.0 Command Line CLI
| MIN(Amount) |
+-----+
| 1000.00 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT Region, SUM(Amount)
-> FROM Sales
-> GROUP BY Region;
+-----+
| Region | SUM(Amount) |
+-----+
| South | 52000.00 |
| North | 1000.00 |
| East | 15000.00 |
+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT Region, SUM(Amount)
-> FROM Sales
-> GROUP BY Region
-> HAVING SUM(Amount) > 100000;
Empty set (0.00 sec)

mysql> SELECT EmpName, Salary
-> FROM Employee E
-> WHERE Salary > (
-> SELECT AVG(Salary)
-> FROM Employee
-> WHERE DeptID = E.DeptID
-> );
+-----+
| EmpName | Salary |
+-----+
| Vijay | 70000.00 |
+-----+
1 row in set (0.00 sec)

mysql> |
```

5. JOIN Operations

-- INNER JOIN

SELECT *

FROM Employee

INNER JOIN Project

ON Employee.EmpID = Project.EmpID;

-- LEFT OUTER JOIN

SELECT *

FROM Employee

LEFT JOIN Project

ON Employee.EmpID = Project.EmpID;

-- RIGHT OUTER JOIN

SELECT *

FROM Employee

RIGHT JOIN Project

ON Employee.EmpID = Project.EmpID;

-- FULL OUTER JOIN

SELECT *

FROM Employee

FULL OUTER JOIN Project

ON Employee.EmpID = Project.EmpID;

```
-- CROSS JOIN
SELECT *
FROM Employee
CROSS JOIN Project;
```

```
mysql> -- INNER JOIN
mysql> SELECT *
  -> FROM Employee
  -> INNER JOIN Project
  -> ON Employee.EmpID = Project.EmpID;
```

EmpID	EmpName	DeptID	Salary	ProjectID	ProjectName	EmpID
1	Rahul	1	60000.00	101	Library System	1
2	Anita	2	45000.00	102	Payroll	2
3	Vijay	1	70000.00	103	Banking App	3

3 rows in set (0.00 sec)

```
mysql>
mysql> -- LEFT OUTER JOIN
mysql> SELECT *
  -> FROM Employee
  -> LEFT JOIN Project
  -> ON Employee.EmpID = Project.EmpID;
```

EmpID	EmpName	DeptID	Salary	ProjectID	ProjectName	EmpID
1	Rahul	1	60000.00	101	Library System	1
2	Anita	2	45000.00	102	Payroll	2
3	Vijay	1	70000.00	103	Banking App	3
4	Sneha	3	55000.00	NULL	NULL	NULL

4 rows in set (0.00 sec)

```
mysql>
mysql> -- RIGHT OUTER JOIN
mysql> SELECT *
  -> FROM Employee
  -> RIGHT JOIN Project
  -> ON Employee.EmpID = Project.EmpID;
```

EmpID	EmpName	DeptID	Salary	ProjectID	ProjectName	EmpID
1	Rahul	1	60000.00	101	Library System	1
2	Anita	2	45000.00	102	Payroll	2

```
  -> ON Employee.EmpID = Project.EmpID;
```

EmpID	EmpName	DeptID	Salary	ProjectID	ProjectName	EmpID
1	Rahul	1	60000.00	101	Library System	1
2	Anita	2	45000.00	102	Payroll	2
3	Vijay	1	70000.00	103	Banking App	3

3 rows in set (0.00 sec)

```
mysql> SELECT *
  -> FROM Employee
  -> FULL OUTER JOIN Project
  -> ON Employee.EmpID = Project.EmpID;
```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '0
UTER JOIN Project
ON Employee.EmpID = Project.EmpID' at line 3

```
mysql>
mysql> -- CROSS JOIN
mysql> SELECT *
  -> FROM Employee
  -> CROSS JOIN Project;
```

EmpID	EmpName	DeptID	Salary	ProjectID	ProjectName	EmpID
1	Rahul	1	60000.00	103	Banking App	3
1	Rahul	1	60000.00	102	Payroll	2
1	Rahul	1	60000.00	101	Library System	1
2	Anita	2	45000.00	103	Banking App	3
2	Anita	2	45000.00	102	Payroll	2
2	Anita	2	45000.00	101	Library System	1
3	Vijay	1	70000.00	103	Banking App	3
3	Vijay	1	70000.00	102	Payroll	2
3	Vijay	1	70000.00	101	Library System	1
4	Sneha	3	55000.00	103	Banking App	3
4	Sneha	3	55000.00	102	Payroll	2
4	Sneha	3	55000.00	101	Library System	1

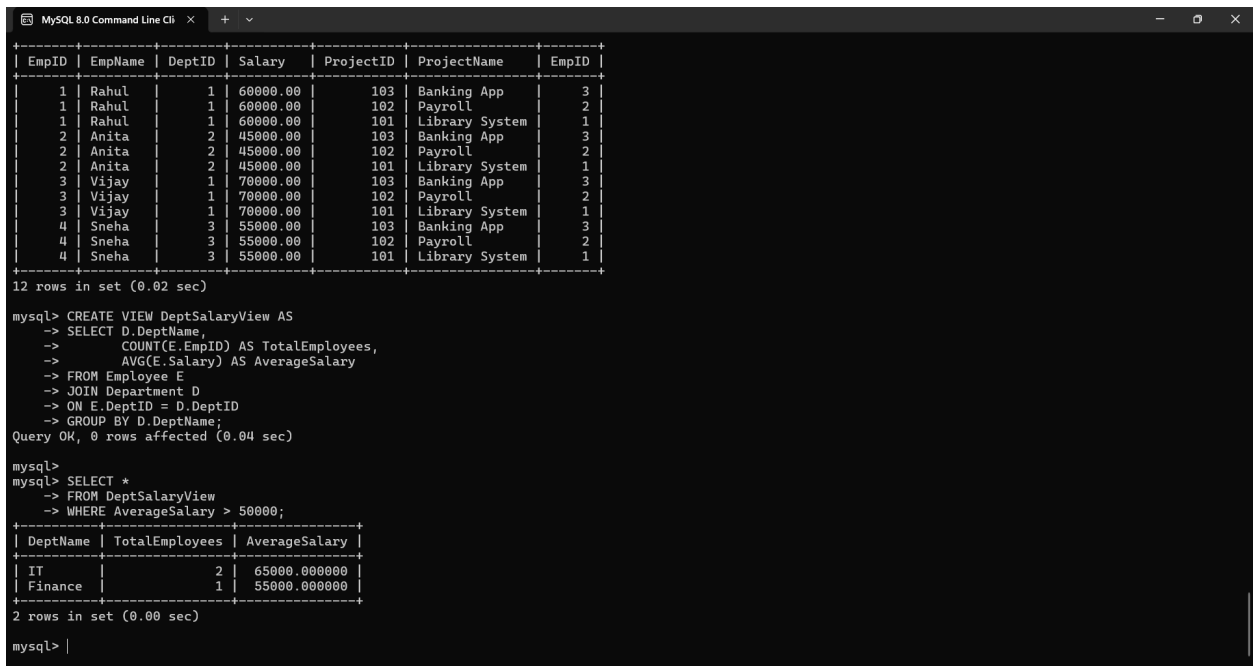
12 rows in set (0.02 sec)

```
mysql> |
```

6. View

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
       COUNT(E.EmpID) AS TotalEmployees,
       AVG(E.Salary) AS AverageSalary
FROM Employee E
JOIN Department D
ON E.DeptID = D.DeptID
GROUP BY D.DeptName;
```

```
SELECT *
FROM DeptSalaryView
WHERE AverageSalary > 50000;
```



```
MySQL 8.0 Command Line Cli x + v
+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary | ProjectID | ProjectName | EmpID |
+-----+-----+-----+-----+-----+-----+
| 1 | Rahul | 1 | 60000.00 | 103 | Banking App | 3 |
| 1 | Rahul | 1 | 60000.00 | 102 | Payroll | 2 |
| 1 | Rahul | 1 | 60000.00 | 101 | Library System | 1 |
| 2 | Anita | 2 | 45000.00 | 103 | Banking App | 3 |
| 2 | Anita | 2 | 45000.00 | 102 | Payroll | 2 |
| 2 | Anita | 2 | 45000.00 | 101 | Library System | 1 |
| 3 | Vijay | 1 | 70000.00 | 103 | Banking App | 3 |
| 3 | Vijay | 1 | 70000.00 | 102 | Payroll | 2 |
| 3 | Vijay | 1 | 70000.00 | 101 | Library System | 1 |
| 4 | Sneha | 3 | 55000.00 | 103 | Banking App | 3 |
| 4 | Sneha | 3 | 55000.00 | 102 | Payroll | 2 |
| 4 | Sneha | 3 | 55000.00 | 101 | Library System | 1 |
+-----+-----+-----+-----+-----+-----+
12 rows in set (0.02 sec)

mysql> CREATE VIEW DeptSalaryView AS
-> SELECT D.DeptName,
-> COUNT(E.EmpID) AS TotalEmployees,
-> AVG(E.Salary) AS AverageSalary
-> FROM Employee E
-> JOIN Department D
-> ON E.DeptID = D.DeptID
-> GROUP BY D.DeptName;
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> SELECT *
-> FROM DeptSalaryView
-> WHERE AverageSalary > 50000;
+-----+-----+-----+
| DeptName | TotalEmployees | AverageSalary |
+-----+-----+-----+
| IT | 2 | 65000.000000 |
| Finance | 1 | 55000.000000 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

7. Relational Algebra Expression

```
[
  \pi_{Name}(Student \bowtie (Enrollment \div Course))
]
```

8. SQL DML Statements

```
INSERT INTO STUDENT
VALUES (101, 'Arun', '2004-03-15', 1);
```

```
UPDATE STUDENT
SET DeptID = 2
WHERE SID = 101;
```

```
DELETE FROM STUDENT
WHERE SID = 101;
```

```
MySQL 8.0 Command Line Cli  x  +  -
+-----+-----+-----+-----+-----+
12 rows in set (0.02 sec)

mysql> CREATE VIEW DeptSalaryView AS
-> SELECT D.DeptName,
->        COUNT(E.EmpID) AS TotalEmployees,
->        AVG(E.Salary) AS AverageSalary
-> FROM Employee E
-> JOIN Department D
-> ON E.DeptID = D.DeptID
-> GROUP BY D.DeptName;
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> SELECT *
-> FROM DeptSalaryView
-> WHERE AverageSalary > 50000;
+-----+-----+-----+
| DeptName | TotalEmployees | AverageSalary |
+-----+-----+-----+
| IT       | 2              | 65000.000000  |
| Finance  | 1              | 55000.000000  |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> INSERT INTO STUDENT
-> VALUES (101, 'Arun', '2004-03-15', 1);
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql>
mysql> UPDATE STUDENT
-> SET DeptID = 2
-> WHERE SID = 101;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> DELETE FROM STUDENT
-> WHERE SID = 101;
Query OK, 1 row affected (0.02 sec)

mysql> |
```

9. Tuple Relational Calculus

$\{t \mid \text{Employee}(t) \wedge t.\text{DeptID} = \text{'IT'} \wedge t.\text{Salary} > 50000\}$

10. Airline Reservation System

```
-- Query 1
SELECT PassengerName, FlightNo
FROM Passenger
JOIN Booking ON Passenger.PassengerID = Booking.PassengerID
JOIN Flight ON Booking.FlightID = Flight.FlightID;
```

```
-- Query 2
SELECT FlightNo
```

```

FROM Flight
WHERE FlightID IN (
    SELECT FlightID
    FROM Booking
);

```

```

-- Query 3
SELECT FlightNo, COUNT(*)
FROM Flight
JOIN Booking ON Flight.FlightID = Booking.FlightID
GROUP BY FlightNo;

```

```

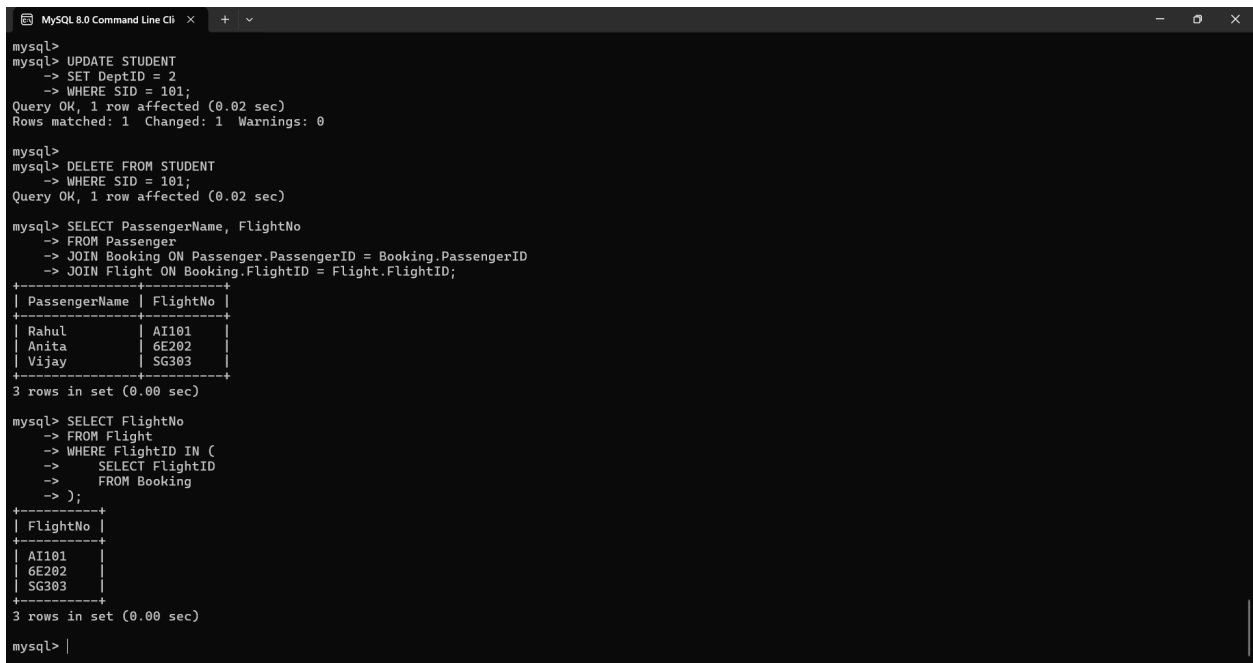
-- Query 4
CREATE VIEW FlightBookingView AS
SELECT PassengerName, FlightNo, SeatNo
FROM Passenger
JOIN Booking ON Passenger.PassengerID = Booking.PassengerID
JOIN Flight ON Booking.FlightID = Flight.FlightID;

```

```

-- Query 5
SELECT *
FROM FlightBookingView;

```



```

MySQL 8.0 Command Line Cli
mysql>
mysql> UPDATE STUDENT
-> SET DeptID = 2
-> WHERE SID = 101;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> DELETE FROM STUDENT
-> WHERE SID = 101;
Query OK, 1 row affected (0.02 sec)

mysql> SELECT PassengerName, FlightNo
-> FROM Passenger
-> JOIN Booking ON Passenger.PassengerID = Booking.PassengerID
-> JOIN Flight ON Booking.FlightID = Flight.FlightID;
+-----+-----+
| PassengerName | FlightNo |
+-----+-----+
| Rahul         | AI101    |
| Anita        | 6E202    |
| Vijay         | SG303    |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT FlightNo
-> FROM Flight
-> WHERE FlightID IN (
->     SELECT FlightID
->     FROM Booking
-> );
+-----+
| FlightNo |
+-----+
| AI101    |
| 6E202    |
| SG303    |
+-----+
3 rows in set (0.00 sec)

mysql>

```



```
MySQL 8.0 Command Line C...
mysql>
mysql> -- Query 4
mysql> CREATE VIEW FlightBookingView AS
  -> SELECT PassengerName, FlightNo, SeatNo
  -> FROM Passenger
  -> JOIN Booking ON Passenger.PassengerID = Booking.PassengerID
  -> JOIN Flight ON Booking.FlightID = Flight.FlightID;
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Query 5
mysql> SELECT *
  -> FROM FlightBookingView;

+-----+-----+-----+
| PassengerName | FlightNo | SeatNo |
+-----+-----+-----+
| Rahul         | AI101    | A1     |
| Anita         | 6E202    | B2     |
| Vijay         | SG303    | C3     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT FlightNo, COUNT(*)
  -> FROM Flight
  -> JOIN Booking ON Flight.FlightID = Booking.FlightID
  -> GROUP BY FlightNo;

+-----+-----+
| FlightNo | COUNT(*) |
+-----+-----+
| AI101    | 1        |
| 6E202    | 1        |
| SG303    | 1        |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> CREATE VIEW FlightBookingView AS
  -> SELECT PassengerName, FlightNo, SeatNo
  -> FROM Passenger
  -> JOIN Booking ON Passenger.PassengerID = Booking.PassengerID
```